General Linear Models

Today's agenda:

- GLMS (9.1 - 9.2)

Previously

We’ve talked about the different steps of the modelling process

- We’ve focused on how to build models from component bits
- E.g., distributions + functional forms + parameters + fitting

Today, we’ll look at an alternative route

General Linear Models

- GLMs among the most common analyses
- Have a standard format of writing (makes reporting easier)
- Pre-built functions (makes writing methods easier)
- Lots of helper functions
- BUT less flexible than MLE approaches

General Linear Models

What is a general linear model?

- Normal error distribution
- Observed values are independent
- Constant variance
- No measurement error
- Linear with respect to the parameters

Linearity in GLMs

$y \sim a + bx^2$

Linear: x^2 is basically just a transformation (like log-transforming)

So still just a parameter multiplied by a predictor variable (hence linear)

$y \sim a + bx^c$

NOT linear: x^c is not a transformed predictor

Predictor variable raised to a parameter, so non-linear

Let's get some data

We'll grab the Avonet data for the parrot family:

```
library(tidyverse)
library(readr)
library(bbmle)
  avo <- read_rds("https://tinyurl.com/avonetdata") %>%
    filter(Family1 == "Psittacidae")
```

Run this and make sure the data load alright

Simple Linear Regression

A.k.a. Ordinary Linear Regression

- Single, continuous predictor variable

Let’s compare the linear regression approach to the MLE approach

Simple Linear Regression

We're going to look at Beak Width vs Beak Depth

plot(avo$Beak.Depth, avo$Beak.Width)

With the MLE approach, this would look like:

```
avo.mle <- mle2(data = avo,
     minuslogl = Beak.Width ~ dnorm(mean = int + a*Beak.Depth,
                                    sd = sd),
     start = list(int = 0,
                  a = 1,
                  sd = 1))
```

Run this now. How do you get a model summary? Model fit look ok?

Simple Linear Regression

The equivalent simple linear regression is:

```
avo.lm <- lm(data = avo,
             formula = Beak.Width ~ Beak.Depth)
```

Run this.

How do we get a summary of this model?

How does the fitted model compare with our mle2 fit?

lm vs mle2 syntax

```
avo.mle <- mle2(data = avo,
       minuslogl = Beak.Width ~ dnorm(mean = int + a*Beak.Depth,
                                      sd = sd),
       start = list(int = 0,
                    a = 1,
                    sd = 1))
avo.lm <- lm(data = avo,
               formula = Beak.Width ~ Beak.Depth)
```

What are the differences you see?

lm vs mle2 syntax

lm:

- Fits intercept and sd automatically (remove intercept with -1)
- Don’t need to specify parameter names (e.g., a*Beak.Depth)
- Don’t need to specify start values (but you can)

lm parameters

To get some of the key parameters:

```
avo.lm$coefficients

avo.lm.summary <- summary(avo.lm)

avo.lm.summary$r.squared

avo.lm.summary$sigma #Equivalent to the SD in our mle2 fit

avo.lm.summary$coefficients
```

Try this out. How does this compare with getting mle2 parameters?

Comparing the approaches

How do the AICs of the two models compare?

Multiple Linear Regression

Extension of simple linear regression with more parameters

- Can be new predictors or powers of existing predictor
- If parameters are powers of predictors: Polynomial regression
    - Or e.g., quadratic for ^2, cubic for ^3, etc.

So if we add in another predictor of Beak Width, it’s a multiple regression

Polynomials in lm

Polynomials are pretty easy in mle2, but have special syntax in lm()

```
avo.mle.poly <- mle2(data = avo,
                minuslogl = Beak.Width ~
                    dnorm(mean = int + a*Beak.Depth + b*Beak.Depth^2,
                           sd = sd),
                  start = list(int = 0,a = 1,b = 0, sd = 1))

avo.lm.poly.1 <- lm(data = avo,
               formula = Beak.Width ~ poly(Beak.Depth,2))

avo.lm.poly.3 <- lm(data = avo,
                    formula = Beak.Width ~ Beak.Depth + I(Beak.Depth^2))
```

Try these three out and compare the fits

Polynomials in lm

Fitting polynomials in R:

`I()` function

- makes R take things “as is”, e.g., `I(Beak.Depth^2)`
- Easier to interpret coefficients
- Can cause issues with correlated predictors

`poly()` function

- add all polynomial terms up to the degree specified
- So, `poly(Beak.Depth)` will give us both Beak.Depth and Beak.Depth^2
- Uses uncorrelated polynomials (better statistically)
- Harder to interpret coefficients

Multiple predictors

Let's try adding Beak Length (nares) to the model:

```
avo.mle.mr <- mle2(data = avo,
                    minuslogl = Beak.Width ~ dnorm(mean = int + a*Beak.Depth +
                    b*Beak.Depth^2 + c*Beak.Length_Nares,
                    sd = sd),
                     start = list(int = 0,
                                  a = 1,
                                  b = 0,
                                  c = 0,
                                  sd = 1))

 avo.lm.mr <- lm(data = avo,
                     formula = Beak.Width ~ poly(Beak.Depth,2) + Beak.Length_Nares)
```

Run these and compare fits

Best model so far?

Use AICtab() to figure out which model(s) are the best so far

Categorical predictors

So far we’ve been using continuous predictors

- What about categorical?

Same basic approach, but then its called an ANOVA rather than a linear model

- ANOVA = analysis of variance

Categorical predictors

We can test whether different habitats have different beak widths

```
avo.lm.anova.1 <- lm(data = avo,
                   formula = Beak.Width ~ Habitat)


avo.lm.anova.2 <- lm(data = avo,
                      formula = Beak.Width ~ Habitat-1)
```

What is the difference between these models?

Categorical predictors

By default, the first category is used as the “intercept”

- Everything else is then relative to that intercept
- Useful if you have one category that is a “control”
    - E.g., you’ll know which values are more/less than the control

Removing the intercept with -1 forces each group to get its own covariate

- Easier to interpret directly
- Statistically identical in terms of model fit

Multi-way ANOVA

ANOVA, but with multiple categorical predictors

```
avo.lm.multianova.1 <- lm(data = avo,
                     formula = Beak.Width ~ Habitat + Trophic.Niche -1 )


avo.lm.multianova.2 <- lm(data = avo,
                          formula = Beak.Width ~ Habitat * Trophic.Niche
                          -1)
```

What is the difference between these two? Which fits better?

Multi-way ANOVA

Understanding ANOVA data:

```
summary(avo.lm.multianova.2)
```

- Tests whether each individual covariate differs from zero

```
anova(avo.lm.multianova.2)
```

- Tests parameter as a whole

```
avo.lm.multianova.2 |> aov() |> TukeyHSD()
```

- Tests pairwise comparisons for all factor levels

Try these and inspect the outputs

What if we have categorical and continuous?

ANCOVA (analysis of covariance)

- Allows intercepts to vary by group
- Allows slopes to vary by group
- Similar syntax to the rest of lm()

Any idea how we would construct a model where:

- Slope of Beak.Width vs Beak.Length varies by trophic niche?
- Each trophic niche has their own intercept?

ANCOVA

```
avo.ancova.1 <- lm(data = avo,
                   formula = Beak.Width ~ Beak.Depth +
                   Trophic.Niche -1 )

avo.ancova.2 <- lm(data = avo,
                   formula = Beak.Width ~ Beak.Depth *
                   Trophic.Niche -1 )
```

How do these models differ in terms of slope and intercept?

Which fits best? What does that mean biologically?

Challenge

Using lm(), try to construct the best model you can of Beak.Width

- Is the best you find an anova,ancova, or lm?
- What parameters did you use?

Before next class:

- Read 9.3 - 9.4